

DPM, DPC, and DPU manual

Ti mmWave Datapath Interface

BY TIM REMMERSWAAL

Contents

1. Introduction	3
1.1. Official TI References	3
2. mmWave Datapath Architecture	4
2.1. DataPath Manager (DPM)	5
API functions	5
Callback functions	5
2.2. Datapath Processing Chain (DPC)	6
API functions	6
Callback functions	6
2.3. Datapath Processing Unit (DPU)	7
API functions	7
2.4. Further details	7
3. Pre-provided functions	8
3.1. Common DPUs	8
3.2. Out-of-the-Box Demo	8
4. Practical Example	9

1. Introduction

This document provides a reference for the software architecture powering Texas Instruments (TI) mmWave radar processors. It outlines the roles and API contracts of the Datapath API and demonstrates how to integrate a custom post-processing DPU into the standard TI Out-of-the-Box (OOB) demo application.

1.1. Official TI References

For more details, examples, and the fundamental foundations of working with TI mmWave software it is recommended to also check out the following resources.

- [TI mmWave SDK Introduction](#) (Video/Presentation): A quick introduction to the TI mmWave SDK, including subsystem structure, drivers, etc.
- [TI mmWave SDK Deep Dive](#) (Video/Presentation): A detailed explanation of the Datapath API, structure of the OOB demo, and advice for how to use them.
- [TI Radar Academy](#) (Portal): A collection of training modules for working with mmWave radar technology. Especially the [Introduction chapter](#) (Text), and the [FMCW Radar Training Series](#) (Video/Presentation) explain the basic principles of mmWave radars really well.
- [TI mmWave SDK User Guide](#) (Text): Detailed SDK documentation that also refers to the relevant Doxygen documentation. Specifically, chapter 5 (MMWAVE SDK deep dive) is useful for the Datapath API interfaces.

2. mmWave Datapath Architecture

The TI mmWave datapath architecture (see Figure 1) consists of three layers. This is done to separate system-level control, pipeline flow management, and core signal processing algorithms and additionally, it abstracts away some of the annoying parts. Each layer of the architecture has expected interfaces and functions, which allows for broad extensibility.

The top layer is the DataPath Manager (DPM), it serves as the system-level orchestration framework responsible for managing execution states across cores (MSS and DSS), handles inter-processor communication (IPC), synchronizing real-time frame interrupts, and dispatching host IOCTL command messages down into the processing pipeline.

The second, and middle, layer is the Datapath Processing Chain (DPC). It functions as the pipeline for a complete radar processing application. The DPC owns the memory layout, coordinates hardware resource allocation, executes the configured Datapath Processing Units across frames or sub-frames, and communicates the final outputs back to the DPM.

The last layer consists of Datapath Processing Units (DPUs), which are standalone processing functions performing some kind of algorithm or mathematical operations. Examples include Range FFT, Doppler FFT, CFAR detection, or Angle-of-Arrival estimation. DPUs are completely decoupled from one another: each DPU consumes standard input memory buffers, writes to output memory buffers, and operates without any dependency or awareness of other DPUs.

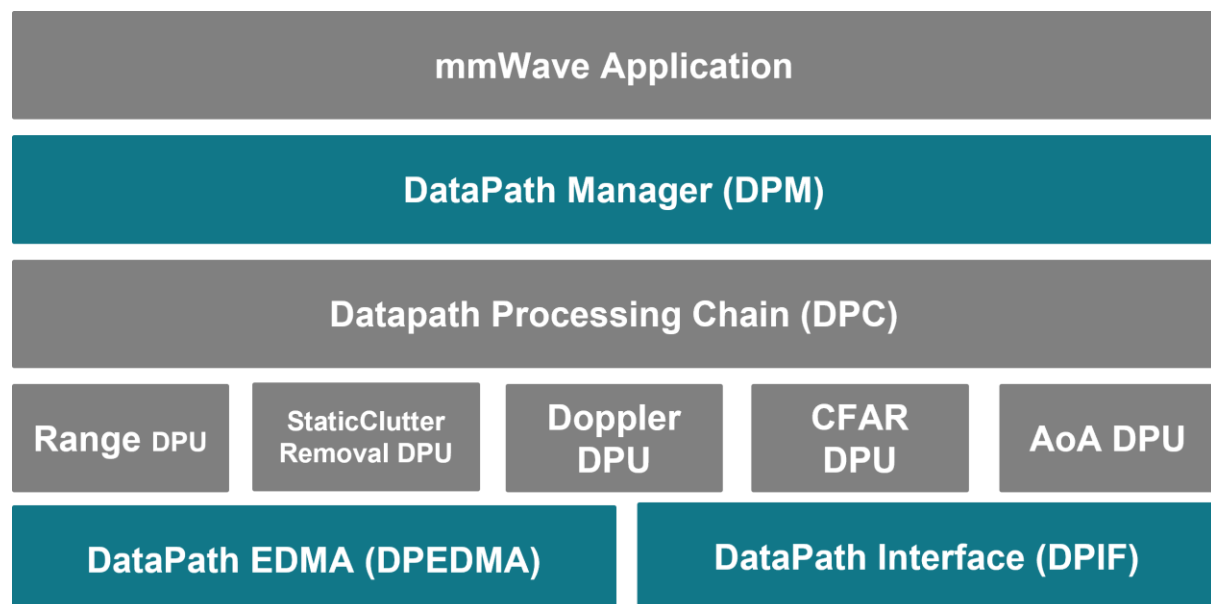


Figure 1 TI mmWave Datapath Architecture

2.1. DataPath Manager (DPM)

The DPM is a TI-provided middleware framework that manages the execution of a DPC. Applications use its API to register a processing chain, perform runtime configurations, and manage radar frame execution. In general, you should not have to modify or customize the DPM.

API functions

- *DPM_init()*: Registers the DPC processing chain with the DPM framework, initializes internal IPC mechanisms, and sets up execution handles.
- *DPM_ioctl()*: Dispatches control configuration commands (IOCTL) from the application or host interface down to the active DPC processing chain.
- *DPM_start()*: Arms hardware radar frame triggers and enables real-time frame processing.
- *DPM_execute()*: Main execution task loop called by the application; blocks until frame/sub-frame processing completes and returns the output result buffer (*DPM Buffer*).
- *DPM_stop()*: Halts real-time frame execution and disables hardware triggers.
- *DPM_deinit()*: De-initializes the DPM framework instance and releases system-level resources.

Callback functions

- *reportFxn*: Invoked by the DPM framework to convey fault codes, assertion failures, and IPC status updates back to the application.
- *processFrameBeginCallBackFxn*: Optional callback triggered immediately upon receiving the RF frame-start hardware interrupt.
- *processInterFrameBeginCallBackFxn*: Optional callback triggered when chirp data acquisition finishes and inter-frame processing begins.

2.2. Datapath Processing Chain (DPC)

The DPC configures and sequences the DPUs required for a complete radar processing application. A DPC implementation can span both the Master Subsystem (MSS) and DSP Subsystem (DSS) cores, though it does not necessarily need all API functions implemented on both sides. For instance, MSS may handle top-level configuration, IOCTL command parsing, and result exporting, while DSS handles real-time DPU pipeline execution and memory allocation.

API functions

- *DPC xxx_init()*: Instantiates child DPU object handles, allocates system control heap memory, and initializes memory pool structures.
- *DPC xxx_start()*: Triggers initial hardware processing (e.g., Range DPU) and arms state structures for frame execution.
- *DPC xxx_execute()*: Sequentially invokes configured DPUs across frame/sub-frame processing periods and packages generated point clouds into output result buffers.
- *DPC xxx_ioctl()*: Receives dynamic and static configuration commands forwarded by *DPM_ioctl()* and routes relevant configuration parameters to target DPUs.
- *DPC xxx_injectData()*: Optional API used to inject external or simulated radar data directly into the processing chain (primarily used for unit testing and offline data validation).
- *DPC xxx_stop()*: Halts pipeline processing, ensuring pending transfers and hardware accelerator tasks are aligned prior to system re-configuration.
- *DPC xxx_deinit()*: Destroys child DPU instance handles and frees allocated DPC control memory.

Callback functions

- *DPC xxx_cbFrameStart()*: ISR callback triggered upon receiving the RF frame-start hardware interrupt; records timing metrics, verifies state consistency, and signals the DPM task thread via *DPM_notifyExecute()*.
- *DPC xxx_cbChirpAvailable()*: Callback triggered upon chirp hardware interrupts, enabling per-chirp processing or chirp-level data ingestion.

2.3. Datapath Processing Unit (DPU)

A DPU contains the algorithmic logic for a specific signal processing function. Every DPU conforms to a standardized TI API, accepting explicit parameter structures that encapsulate hardware resources, static operational settings, dynamic runtime configurations, and execution performance metrics.

API functions

- *DPU xxx init()*: Allocates internal instance control memory, initializes object handles, and binds hardware resources (e.g., HWA, EDMA).
- *DPU xxx config()*: Binds execution resources, input/output memory pointers, and operational parameters (hardware resources, static frame settings, and dynamic controls) to the DPU handle.
- *DPU xxx process()*: Core real-time execution entry point; performs mathematical and signal processing operations on input buffers and writes results and statistics to output buffers.
- *DPU xxx control()*: Executes dynamic control commands to update runtime parameters or reset internal stats without requiring buffer re-allocation or full chain reconfiguration.
- *DPU xxx deinit()*: Releases allocated DPU control instances and frees internal memory handles.

2.4. Further details

For the detailed descriptions of the functions the mmWave SDK User Guide is the recommended starting place:

- Ch. 5.4.7 discusses the DPU
- Ch. 5.4.8 discusses the DPM
- Ch. 5.4.9 discusses the DPC.

These chapters also contain references to the Doxygen documentation that can be found in the local install folder of the SDK.

3. Pre-provided functions

TI provides a set of hardware-optimized DPUs targeting either the Hardware Accelerator (HWA) or the C674x DSP core, depending on the specific mmWave IC architecture (e.g., AWR1843, AWR6843, AWR2944). These DPUs encapsulate standard radar processing algorithms and conform to the standardized DPU interface.

3.1. Common DPUs

Table 1 lists the common pre-provided DPUs available in the TI mmWave SDK along with their primary processing functions, and I/O data formats. Most of these have an HWA and DSP implementation available.

Table 1 Common SDK Datapath Processing Units (DPUs)

DPU Module	Processing Function	Input Data	Output Data
RangeProc	1D FFT across time-domain ADC samples	ADC Buffer	Radar Cube
StaticClutterProc	DC mean subtraction across Doppler chirps	Radar Cube	Radar Cube
DopplerProc	2D FFT across chirps	Radar Cube	Detection Matrix
CFARCAProc	Peak detection (CFAR-CA, CFAR-SO, CFAR-GO)	Detection Matrix	CFAR SNR Point List
AoAProc	3D Angle-of-Arrival estimation & phase calibration	Radar Cube + CFAR List	Cartesian Point Cloud (x,y,z)

3.2. Out-of-the-Box Demo

The Out-of-the-Box (OOB) demo serves as the standard reference application for TI mmWave processors, demonstrating how common DPUs are integrated into a functional Datapath Processing Chain implemented across [*objectdetection.c*](#) and [*objectdetectioninternal.h*](#). The application utilizes both the Master Subsystem (MSS) and DSP Subsystem (DSS) cores. The MSS manages system initialization, serial interface tasks, and host communications, while the DSS handles high-performance radar signal processing and DPU pipeline execution.

A UART CLI interface allows the application to process runtime configuration files, such as those generated by the [TI mmWave Demo Visualizer](#), to set chirp profiles, frame timing, and algorithm parameters dynamically. Following frame execution, the application formats point cloud outputs and diagnostic metadata into serialized data packets, which are streamed back to external visualizers over a UART interface.

4. Practical Example

This chapter presents a reference implementation of a custom DPU (*DPU Custom*) targeting the AWR1843-Boost evaluation board, compiled with the C89 TI compiler. The custom unit performs spatial filtering on incoming Cartesian point cloud targets based on configurable lateral and depth distance thresholds. Within the valid target region, the unit evaluates target distances to retain up to three representative objects: the closest, middle (median), and furthest detected points.

The custom unit is implemented in *dpu_custom.c* and *dpu_custom.h*, strictly following the standard *DPU xxx* interface (init, config, process, control, and deinit). Control memory is allocated on startup and freed during teardown, while inter-frame processing evaluates candidate targets, isolates representative distance metrics, and measures processing execution time. Dynamic spatial thresholds are managed through *DPU Custom control()* using dedicated command enums (*DPU Custom Cmd*), enabling runtime parameter updates without reallocating memory.

Using the SDK Out-of-the-Box (OOB) demo as a starting point, integrating *DPU Custom* into the Datapath Processing Chain (*objectdetection.c*) follows the standard TI orchestration model as described in Chapter 2. A local duplicate of *objectdetectioninternal.h* from the OOB demo source is modified to include the custom DPU handle and configuration structures within the internal DPC object handle. The custom DPU handle is instantiated during *DPC ObjectDetection init()*, configured with input pointers from *AoAProc* and L3 RAM output buffers during *DPC ObjectDetection config()*, and executed within *DPC ObjectDetection execute()* immediately following angle-of-arrival estimation. The filtered Cartesian point cloud is then streamed to the UART output.

For the complete code and modifications see the [source code \(Github\)](#).